Student Reflection Notes: Cloud Computing Mini Project

Name: Dixant Soni | Roll No: 230103029 | Course: Cloud Computing (CS3081)

GitHub: https://github.com/DIXANTOFFICIAL1/Knowledge-Distillation-and-Deployment-of-a-Lightweight-

1. Executive Learning & Paradigm Shifts

*** Sequence-Level Knowledge Distillation on Edge/Cloud:**

Prior to this assignment, I viewed LLMs primarily through the lens of API consumers or multi-GPU training. Distilling phi-3.5 (3.8B parameters) into pythia-70m on pure CPU hardware demonstrated the immense value of task-specific sequence distillation. The student does not need the teacher's full parameter volume; by matching discrete target sequences formatted as strict 4-key JSON, the student achieved 70% full-frame accuracy while shrinking parameter count by 54x.

*** Decoupling Storage from Ephemeral Compute:**

Storing model weights directly inside Docker container images is an anti-pattern that bloats image transfer sizes and violates 12-factor cloud principles. By establishing SeaweedFS as an S3-compatible in-cluster object store and downloading weights dynamically during application initialization (/readyz gating), I learned how enterprise cloud infrastructures achieve rapid microservice updates without redeploying container registries.

*** Kubernetes Health Probe Synchronization:**

A vital learning was probe orchestration. Setting up /healthz (liveness) to return 200 immediately while /readyz (readiness) returns 503 during model weight downloads prevented Traefik Ingress from directing load test traffic to unready pods, eliminating 502/504 Bad Gateway errors entirely during cold starts.

2. Key Engineering Challenges Faced & Solutions Implemented

*** Challenge 1: Ollama phi3.5 CPU Throughput & Labeling 3,000 Commands**

Problem: Running phi3.5-mini-instruct locally on a single CPU thread to label 3,000 commands projected an execution time of over 6.5 hours, threatening project deadlines.

Solution: I engineered a stateless worker sharding architecture in worker/worker.py. By spawning multiple parallel worker pods in the 'pipeline' namespace using a deterministic modulo strategy (line_index % SHARD_COUNT == SHARD_INDEX), the workload was partitioned seamlessly with zero overlapping records and zero gaps.

*** Challenge 2: PyTorch Container Image Size Constraints (< 2 GB)**

Problem: Standard PyTorch wheels bundle extensive CUDA runtime libraries, producing Docker images in excess of 6.2 GB, far exceeding the strict 2.0 GB submission limit.

Solution: I leveraged PyTorch's specialized CPU-only wheel repository (--extra-index-url https://download.pytorch.org/whl/cpu) inside a multi-stage python:3.10-slim Dockerfile. This reduced the final image size to 842 MB (a 86% reduction) while satisfying all runtime dependencies.

*** Challenge 3: Stateless NACL Asymmetric Return Traffic Blockage**

Problem: In Task 5, initial packet simulations dropped outbound client replies despite valid inbound HTTP rules.

Solution: Unlike stateful security groups, stateless NACLs require explicit return rules. I added outbound rules for ephemeral port ranges (1024-65535) and placed the malicious subnet deny rule (198.51.100.0/24) at rule number 100, ensuring malicious traffic is dropped before permissive web rules are evaluated.

3. Observations, Findings & Architectural Improvements

*** Observation: The Pareto Frontier of Compressed Models**

Evaluating pythia-70m on validation.jsonl revealed that while JSON validity reached 96.0% and intent accuracy was 96.0%, date slot extraction dropped to 76.0%. Small models excel at learning deterministic syntax and broad categorical classification, but struggle with fuzzy span boundaries (e.g. distinguishing 'tomorrow' vs 'the weekend after next').

*** Resource Utilization & Operational Costs:**

During the 2-minute Locust load test with 4 concurrent users, memory consumption stabilized at 486.2 MiB (peak), well within the 1 GiB limit. Under the given rate card (3.50 INR/vCPU-hr, 0.45 INR/GiB-hr), serving 1,000 requests cost approximately 0.3657 INR, demonstrating the massive economic benefit of lightweight specialized SLMs.

*** Future Architectural Improvements:**

1. Model Optimization: Incorporate INT8 quantization via ONNX Runtime or GGML to achieve sub-50ms CPU latency. 2. Semantic Caching: Introduce an in-memory Redis cache for frequently queried commands (e.g., weather and alarms) to bypass inference entirely for 40%+ of voice traffic. 3. Multi-Cluster Resiliency: Extend k3d single-node orchestration to a multi-node production Kubernetes cluster with cross-region failover.